BLOCKSEC

Security Audit Report for DStock Solana OFT Programs

Date: January 30, 2026 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction **1**

1.1 About the Audit Target . . . 1

1.2 Disclaimer . . . 1

1.3 Procedure of Auditing . . . 2

1.3.1 Security Issues . . . 2

1.3.2 Additional Recommendation . . . 3

1.4 Security Model . . . 3

Chapter 2 Findings **4**

2.1 Security Issue . . . 4

2.1.1 Incompatibility between the OFT mechanism and the `DStockWrapper` token accounting system . . . 4

2.1.2 Permanent loss of control due to lack of zero address validation in configuration updates . . . 5

2.2 Recommendation . . . 7

2.2.1 Allow setting multiple peer config fields per invocation . . . 7

2.3 Note . . . 8

2.3.1 Solana version should also use a blacklist validation mechanism . . . 8

2.3.2 `LayerZero` endpoint configuration . . . 8

2.3.3 Set peers correctly for each OFT contract . . . 9

2.3.4 OFTAdapter should only be deployed in only one chain . . . 9

2.3.5 Potential centralization risks . . . 9

2.3.6 Decimal precision loss in OFT transfers . . . 9

2.3.7 Updating capacity refills `tokens` in `set_capacity()` . . . 10

2.3.8 Correct implementation and configuration assumptions for Solana OFT bridging . . . 10

Report Manifest

Item	Description
Client	DStock Official
Target	DStock Solana OFT Programs

Version History

Version	Date	Description
1.0	January 30, 2026	First release

Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-01-30 10:11:22+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About the Audit Target

Information	Description
Type	Smart Contract
Language	Rust, Solidity
Approach	Semi-automatic and manual verification

The audit target (hereinafter referred to as the Target) of this audit is the code repository [1] of DStock Solana OFT Programs of DStock Official.

This project defines the initialization logic for an OFT program on Solana using the Anchor framework. The InitOFT instruction creates and configures the core OFTStore account, a token escrow account, and auxiliary LzReceiveTypesAccounts, all derived with deterministic PDA seeds. During initialization, it validates token decimals, computes the local-to-shared decimal conversion rate, sets administrative and fee parameters, and records the token mint and escrow addresses. The instruction also registers the OFT as an OApp with a LayerZero endpoint via CPI. Supporting types define OFT modes (Native or Adapter) and utility functions for amount conversion and dust handling.

Note this audit only focuses on the smart contracts in the following directories/files:

- solana/

Other files are not within the scope of the audit. Additionally, all dependencies of the Target are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
DStock Solana OFT Programs	`Version 1`	`761919d05db6ceb2377e2609923a41bb85704b29`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

[1] https://github.com/dstockofficial/dstock-oft-hype

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the Target, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of the Target.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan the Target with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of the Target and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism
* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization
* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	Likelihood: *High*	Likelihood: *Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

Chapter 2 Findings

In total, we found **two** potential security issues. Besides, we have **one** recommendation and **eight** notes.

- Low Risk: 2
- Recommendation: 1
- Note: 8

ID	Severity	Description	Category	Status
1	Low	Incompatibility between the OFT mechanism and the `DStockWrapper` token accounting system	Security Issue	Confirmed
2	Low	Permanent loss of control due to lack of zero address validation in configuration updates	Security Issue	Confirmed
3	-	Allow setting multiple peer config fields per invocation	Recommendation	Confirmed
4	-	Solana version should also use a blacklist validation mechanism	Note	-
5	-	`LayerZero` endpoint configuration	Note	-
6	-	Set peers correctly for each OFT contract	Note	-
7	-	OFTAdapter should only be deployed in only one chain	Note	-
8	-	Potential centralization risks	Note	-
9	-	Decimal precision loss in OFT transfers	Note	-
10	-	Updating capacity refills `tokens` in `set_capacity()`	Note	-
11	-	Correct implementation and configuration assumptions for Solana OFT bridging	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Incompatibility between the OFT mechanism and the `DStockWrapper` token accounting system

Severity Low

Status Confirmed

Introduced by `Version 1`

Description The OFT implements a cross-chain transfer mechanism that utilizes the function `apply()` to synchronize token amounts across different networks. The contract `DStockWrapper` employs a dual-variable accounting system where the internal ledger tracks shares rather than raw amounts, which results in a dynamic exchange rate that fluctuates over time. Because the function `apply()` only facilitates the transfer of absolute amounts without accounting for the

underlying shares, the system fails to reconcile the specific share values that are required for accurate bookkeeping.

```
impl LzReceive<'_> {
  pub fn apply(ctx: &mut Context<LzReceive>, params: &LzReceiveParams) -> Result<()> {
      require!(!ctx.accounts.oft_store.paused, OFTError::Paused);

      let oft_store_seed = ctx.accounts.token_escrow.key();
      let seeds: &[&[u8]] = &[OFT_SEED, oft_store_seed.as_ref(), &[ctx.accounts.oft_store.bump]];

      // Validate and clear the payload
      let accounts_for_clear = &ctx.remaining_accounts[0..Clear::MIN_ACCOUNTS_LEN];
      let _ = oapp::endpoint_cpi::clear(
          ctx.accounts.oft_store.endpoint_program,
          ctx.accounts.oft_store.key(),
          accounts_for_clear,
          seeds,
          ClearParams {
              receiver: ctx.accounts.oft_store.key(),
              src_eid: params.src_eid,
              sender: params.sender,
              nonce: params.nonce,
              guid: params.guid,
              message: params.message.clone(),
          },
      )?;

      // Convert the amount from sd to ld
      let amount_sd = msg_codec::amount_sd(&params.message);
      let mut amount_received_ld = ctx.accounts.oft_store.sd2ld(amount_sd);
```

Listing 2.1: solana/programs/oft/src/instructions/lz_receive.rs

Impact This discrepancy leads to critical accounting errors within the contract `DStockWrapper` as the cross-chain transaction relies on outdated amount data instead of the necessary share-based calculations.

Suggestion Revise the logic accordingly.

Feedback from the project The project team indicated they will strictly ensure a 1:1 ratio between shares and amount.

2.1.2 Permanent loss of control due to lack of zero address validation in configuration updates

Severity Low

Status Confirmed

Introduced by `Version 1`

Description The function `apply()` allows the admin to modify protocol parameters (e.g., `admin`, `delegate`, `pauser` and `unpauser`). The implementation lacks necessary validation to ensure that

these newly assigned addresses are not the zero address. Assigning the `admin` to the zero address results in the permanent loss of control over management functions since no entity can produce a valid signature for the null address.

```rust
#[derive(Accounts)]
pub struct SetOFTConfig<'info> {
    pub admin: Signer<'info>,
    #[account(
        mut,
        seeds = [OFT_SEED, oft_store.token_escrow.as_ref()],
        bump = oft_store.bump,
        has_one = admin @OFTError::Unauthorized
    )]
    pub oft_store: Account<'info, OFTStore>,
}

impl SetOFTConfig<'_> {
    pub fn apply(ctx: &mut Context<SetOFTConfig>, params: &SetOFTConfigParams) -> Result<()> {
        match params.clone() {
            SetOFTConfigParams::Admin(admin) => {
                ctx.accounts.oft_store.admin = admin;
            },
            SetOFTConfigParams::Delegate(delegate) => {
                let oft_store_seed = ctx.accounts.oft_store.token_escrow.key();
                let seeds: &[&[u8]] =
                    &[OFT_SEED, &oft_store_seed.to_bytes(), &[ctx.accounts.oft_store.bump]];
                let _ = oapp::endpoint_cpi::set_delegate(
                    ctx.accounts.oft_store.endpoint_program,
                    ctx.accounts.oft_store.key(),
                    &ctx.remaining_accounts,
                    seeds,
                    SetDelegateParams { delegate },
                )?;
            },
            SetOFTConfigParams::DefaultFee(fee_bps) => {
                require!(fee_bps < MAX_FEE_BASIS_POINTS, OFTError::InvalidFee);
                ctx.accounts.oft_store.default_fee_bps = fee_bps;
            },
            SetOFTConfigParams::Paused(paused) => {
                ctx.accounts.oft_store.paused = paused;
            },
            SetOFTConfigParams::Pauser(pauser) => {
                ctx.accounts.oft_store.pauser = pauser;
            },
            SetOFTConfigParams::Unpauser(unpauser) => {
                ctx.accounts.oft_store.unpauser = unpauser;
            },
        }
        Ok(())
    }
}
```

Listing 2.2: solana/programs/oft/src/instructions/set_oft_config.rs

Impact Assigning the `admin` to the zero address results in the permanent loss of control over management functions because the required signatures can no longer be provided.

Suggestion Revise the logic accordingly.

Feedback from the project The project team stated this is `LayerZero`'s official scaffold code and they will not modify it, but will ensure the zero address is never set.

2.2 Recommendation

2.2.1 Allow setting multiple peer config fields per invocation

Status Confirmed

Introduced by `Version 1`

Description The function `apply()` updates the peer state by matching on the input `params.config`, which is a single `PeerConfigParam` variant. This design enforces that only one configuration field (e.g., `peer.peer_address`) can be modified per invocation. As a result, configuring multiple fields requires multiple transactions, which prevents atomic updates and may temporarily leave the peer account in an intermediate or inconsistent configuration. It is recommended to allow setting multiple peer config fields per invocation.

```
pub fn apply(ctx: &mut Context<SetPeerConfig>, params: &SetPeerConfigParams) -> Result<()> {
    match params.config.clone() {
        PeerConfigParam::PeerAddress(peer_address) => {
            ctx.accounts.peer.peer_address = peer_address;
        },
        PeerConfigParam::FeeBps(fee_bps) => {
            if let Some(fee_bps) = fee_bps {
                require!(fee_bps < MAX_FEE_BASIS_POINTS, OFTError::InvalidFee);
            }
            ctx.accounts.peer.fee_bps = fee_bps;
        },
        PeerConfigParam::EnforcedOptions { send, send_and_call } => {
            oapp::options::assert_type_3(&send)?;
            ctx.accounts.peer.enforced_options.send = send;
            oapp::options::assert_type_3(&send_and_call)?;
            ctx.accounts.peer.enforced_options.send_and_call = send_and_call;
        },
        PeerConfigParam::OutboundRateLimit(rate_limit_params) => {
            Self::update_rate_limiter(
                &mut ctx.accounts.peer.outbound_rate_limiter,
                &rate_limit_params,
            )?;
        },
        PeerConfigParam::InboundRateLimit(rate_limit_params) => {
            Self::update_rate_limiter(
                &mut ctx.accounts.peer.inbound_rate_limiter,
                &rate_limit_params,
            )?;
        },
    }
```

```
    ctx.accounts.peer.bump = ctx.bumps.peer;
    Ok(())
}
```

Listing 2.3: solana/programs/oft/src/instructions/set_peer_config.rs

Suggestion Revise the logic accordingly.

Feedback from the project The project team stated that the Solana version will only set a single destination chain and configure it once, so the impact is limited in the current usage scenario.

2.3 Note

2.3.1 Solana version should also use a blacklist validation mechanism

Introduced by `Version 1`

Description On the EVM side, the contract `DStockOFTUpgradeable` implements a comprehensive blacklist management mechanism via `updateBlackList()`. The Solana version should also include equivalent on-chain blacklist state storage or transfer-time validation logic to maintain parity in access control.

```
function updateBlackList(address _user, bool _isBlackListed) external onlyRole(
    BLACKLIST_MANAGER_ROLE) {
    if (_user == address(0)) revert InvalidAddress(_user);
    if (_user == address(this)) revert InvalidAddress(_user);
    if (_user == owner()) revert InvalidAddress(_user);
    if (_user == treasury) revert InvalidAddress(_user);
    if (hasRole(DEFAULT_ADMIN_ROLE, _user)) revert InvalidAddress(_user);

    blackList[_user] = _isBlackListed;
    emit BlackListUpdated(_user, _isBlackListed);
}
```

Listing 2.4: evm/src/DStockOFTUpgradeable.sol

Feedback from the project The project team indicated that an internal version with a blacklist mechanism exists.

2.3.2 `LayerZero` endpoint configuration

Introduced by `Version 1`

Description In this project, each chain must be configured with the correct `LayerZero` endpoint address to ensure proper functionality and communication. Incorrect management of these endpoint addresses could lead to operational failures or vulnerabilities in the protocol.

```
    ctx.accounts.oft_store.endpoint_program =
        if let Some(endpoint_program) = params.endpoint_program {
            endpoint_program
        } else {
```

```
            ENDPOINT_ID
        };
```

Listing 2.5: solana/programs/oft/src/instructions/init_oft.rs

2.3.3 Set peers correctly for each OFT contract

Introduced by `Version 1`

Description To ensure successful one-way messages between chains, it's essential to establish peer configurations by invoking the functions `set_peer_config()` on solana. Both source and destination chains' OFTs perform peer verification before executing the message on the destination chain, ensuring secure and reliable cross-chain communication.

```
                ctx.accounts.oft_store.key(),
                &ctx.remaining_accounts,
                seeds,
```

Listing 2.6: solana/programs/oft/src/instructions/set_oft_config.rs

2.3.4 OFTAdapter should only be deployed in only one chain

Introduced by `Version 1`

Description There can only be one OFTAdapter in the omnichain deployment. Multiple adapters break unified liquidity and can cause permanent token loss due to insufficient destination supply.

2.3.5 Potential centralization risks

Introduced by `Version 1`

Description In this project, several privileged roles (e.g., `admin`) can conduct sensitive operations, which introduces potential centralization risks. For example, the `admin` could set peers for OFT contracts based on the protocol. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

2.3.6 Decimal precision loss in OFT transfers

Introduced by `Version 1`

Description Due to the shared decimal precision of OFT being 6, any fractional units smaller than `1 / ld2sd_rate` become "dust", where `ld2sd_rate` is `10u64.pow((ctx.accounts.token_mint.decimals - params.shared_decimals)`. By removing dust, the protocol should guarantee that every bridged amount fits within the shared-decimal limits of all chains. That leftover dust is returned to the sender. Meanwhile, the protocol should remove it after fees and before bridging to avoid accidentally rounding away more than intended.

2.3.7 Updating capacity refills `tokens` in `set_capacity()`

Introduced by `Version 1`

Description The program state `PeerConfig` relies on token bucket accounting to enforce rate limits, the function `set_capacity()` currently assigns the variable `tokens` to the full value of the variable `capacity`. The variable `tokens` represent available inbound or outbound capacity. When an administrator reduces `capacity` to constrain throughput, since the update refills the variable `tokens` to the new limit at the moment of change. For example, if the current `capacity` is 100 and `tokens` are 30, then an administrator reduces `capacity` to 60, `set_capacity()` resets tokens to 50, thus the actual allowance increases from 30 to 50, it is noted that the reduction grants a fresh burst allowance rather than tightening allowance immediately.

2.3.8 Correct implementation and configuration assumptions for Solana OFT bridging

Introduced by `Version 1`

Description In the project, the bridging process relies on the correct implementation and configuration of `OFT` or `OFTAdapter` used for both the wrapped tokens and the underlying tokens.

The related cross chain logic and architectural assumptions are described as follows:

1. Solana is treated as the source chain, while HyperEVM is used as the destination chain for cross chain transfers. It is assumed that the underlying cross chain messaging mechanism can correctly relay messages.

2. A 1:1 mapping relationship is adopted between `share` and `amount`, meaning that no ratio conversion or rebasing logic is involved during the cross chain transfer process.

3. The assets on BSC and Solana are not the same wrapped tokens. Therefore, their corresponding `OFT` or `OFTAdapter` contracts are considered independent from each other, and no unified fungibility or supply consistency across different chains is assumed.

4. Two `OFTAdapter` contracts are deployed on the two source chains to support cross chain inflows from different source assets (BSC and Solana). It is assumed that each `OFTAdapter` is correctly configured and mapped to its corresponding target chain asset representation, thereby ensuring the correct minting and accounting of the corresponding OFT tokens on HyperEVM.

5. It is assumed that all critical parameter configurations (e.g., peers for supported paths, and endpoint address for access control) have been correctly set, ensuring that cross chain transfers can be properly routed and executed.

BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS